Project 16 (Real-World Example) — JobMatch-Resume-To-Job-Description-Skill-Matcher

Type: RAG pipeline (extract → retrieve → match → report) · **Stack:** Python, LangChain, Chroma (RAG), `langchain-community` loaders, OpenAI SDK, Pydantic · **Level:** Intermediate

Tagline: Match a resume against a job description by *meaning*, not keywords — pull the required skills out of the posting, search the resume for each one with RAG, and report a match score plus the exact gaps.

Charter (what this project is)

A recruiter has one job description and a stack of resumes, and "does this person fit?" takes real time to answer well — a keyword search misses "JS" when the posting says "JavaScript". **JobMatch** does it by meaning: it extracts the required skills from the job description, indexes the resume with RAG, and for each required skill searches the resume semantically. A skill the resume covers (even in different words) counts as matched; one it doesn't is a gap. The output is a **match score** and the **missing skills** — the two things a recruiter actually wants.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3

The Story — what you're building

A job description and a resume come in. A **skill extractor** reads the posting and returns a typed list of required skills. A **RAG layer** chunks the resume, embeds it, and stores it in Chroma. Then the **matcher** takes each required skill and searches the resume for it: if the closest resume chunk is near enough (distance below a cutoff), the skill is **matched**; if not, it's **missing**. A deterministic step turns matched/missing into a **score** and a gap list, and a formatter prints the report.

The lesson: matching by *meaning* is a RAG problem, not a string problem. "Built REST APIs in Python" should match a "Python" requirement, and a keyword check can't see that — but a semantic search can. The matcher stays deterministic (a distance below a cutoff is a match), so you can test the whole decision without a model.

Project Structure (the professional layout)

`main.py` takes a resume file and a job-description file; all code in `src/`.

```
<repo folder>/ (the JobMatch repo — full name in the heading above)
├── samples/
│   ├── job.txt                a job description
│   └── resume.txt             a candidate resume
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── resume_store.py    build_resume_store()      (Step 2)
│   │   └── retriever.py       best_distance()           (Step 2)
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   ├── extractor.py           extract_required_skills() (Step 2)
│   ├── matcher.py             match_skills()            (Step 1)
│   ├── pipeline.py            run_match()               (Step 3)
│   └── schemas.py             RequiredSkills, MatchReport (Step 1)
├── tests/
│   ├── integration/test_live_match.py
│   └── unit/                  test_config, test_matcher
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds the standard setup, `src/schemas.py` and `src/matcher.py` (the deterministic matcher) with tests — using a fake retriever, no model or embeddings.
- **Step 2** adds `src/rag/` (chunk + embed + store + retrieve the resume) and `src/extractor.py` (pull required skills from the JD).
- **Step 3** adds `src/pipeline.py`, `main.py`, and the integration test.

Setup (do this once, before Step 1)

```
cd JobMatch-Resume-To-Job-Description-Skill-Matcher
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langchain-openai langchain-community langchain-text-splitters \
    chromadb openai pydantic python-dotenv
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. Write a short `samples/resume.txt` and `samples/job.txt` with a few overlapping and a few non-overlapping skills. `LICENSE` is the MIT text.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts; this builds on Doc05/08, not pytest):

- **`tests/unit/`** — no model, no embeddings: `test_config`; `test_matcher` (the matcher with a **fake** `distance(skill)` function — a near distance is matched, a far one is missing, and the score is the right percentage).
- **`tests/integration/test_live_match.py`** — real model + embeddings: a resume that clearly covers most of a job's skills scores high and names the one gap.

Each step ends with a **Tests for this step** box.

Step 1 — The Match Report and the Matcher

What this step does: the typed shapes and the deterministic `match_skills` — the decision engine — built and tested with a **fake** retriever, no model or embeddings.

Why this step matters: the whole "does this person fit" answer lives here. Because `match_skills` takes a `distance(skill)` function, you can test every path — matched, missing, the score — with made-up distances, before any real search exists.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write the standard setup (`config` / `exceptions` / `utils/logger` / `models/llm`). 2. Write `src/schemas.py`: `RequiredSkills(skills: list)` (the extractor's output) and `MatchReport(score, matched, missing)`. 3. Write `src/matcher.py`: `match_skills(required, distance, cutoff=0.8) -> MatchReport` — for each required skill, call `distance(skill)`; `<= cutoff` is matched, else missing; `score` is the matched percentage (0 when there are no required skills). `distance` is a callable you pass in.

Tests for this step: `test_matcher` — with a fake `distance` (near for "python", far for "kubernetes"), the right skills land in matched/missing and the score is correct. `test_config`.

Step 2 — The RAG Layer and the Skill Extractor

What this step does: the resume RAG store (chunk + embed + retrieve) and the job-description skill extractor.

Why this step matters: this is where "by meaning" happens — the resume is embedded so a skill can be found even when it's phrased differently, and the JD's requirements are pulled out as a clean typed list.

What's new vs. Step 1: `src/rag/*`, `src/extractor.py`. **What stays the same:** the matcher and schemas.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. `src/rag/resume_store.py`: `build_resume_store(resume_text)` — split the resume (Doc08's `RecursiveCharacterTextSplitter`), embed with `OpenAIEmbeddings`, add to a Chroma collection. 2. `src/rag/retriever.py`: `best_distance(collection, skill)` — query for the skill, return the closest chunk's distance (or a big number if empty). 3. `src/extractor.py`: `extract_required_skills(model, job_text)` = prompt | `model.with_structured_output(RequiredSkills)`.

Tests for this step: proven live in Step 3; for now, `best_distance` returns a small number for a skill the resume covers and a large one for a skill it doesn't.

Step 3 — The Pipeline (final)

What this step does: `run_match` ties it together — build the resume store, extract the JD skills, match each one, return the report — plus `main.py` and the integration test.

Why this step matters: this is the one call a caller makes, and it's just the three pieces in order. The matcher's `distance` argument becomes `lambda skill: best_distance(collection, skill)`.

What's new vs. Step 2: `src/pipeline.py`, `main.py`. **What stays the same:** everything else.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/pipeline.py`: `run_match(model, resume_text, job_text)` — build the store, extract required skills, call `match_skills(required, lambda s: best_distance(collection, s))`, return the `MatchReport`. 2. Write `main.py` (read both files, print the score, matched, and missing) and `tests/integration/test_live_match.py`.

Final Deliverable: a system that scores a resume against a job description by meaning and names the missing skills.

Tests for this step: `test_live_match` — a resume covering most of a job's skills scores high and lists the one real gap.

Run, Test and Configure the Final Project

```
python main.py samples/resume.txt samples/job.txt
python -m tests.unit.test_config
python -m tests.unit.test_matcher
python -m tests.integration.test_live_match
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity
`MATCH_CUTOFF`	no	`0.8`	Max distance that still counts as a match

Checklist Before You Call This Done

- [ ] Matching is semantic (RAG), so "Python" matches "built APIs in Python"
- [ ] `match_skills` takes a `distance` function, so it's testable with no model
- [ ] The score is a percentage, and 0 required skills doesn't divide by zero
- [ ] The report names the exact missing skills, not just a number
- [ ] You can explain why this is a RAG problem and not a keyword search

Publish It to GitHub

Name the repo `JobMatch-Resume-To-Job-Description-Skill-Matcher`, add the `README.md`, push. A semantic resume matcher is a portfolio piece every recruiter and engineer gets instantly.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** before the solution. Build the matcher (Step 1) first — it's the decision engine, and it needs no model.